2 marks

1.Define a biological neuron. List its main components.

Biological Neuron:
A biological neuron is a nerve cell that receives, processes, and transmits information in the form of electrical and chemical signals in the nervous system.

Main components:

1. Dendrites – receive signals
2. Cell body (Soma) – processes signals
3. Axon – transmits signals
4. Synapse – passes signals to the next neuron

2. Computational Neuron:
A computational neuron is a mathematical model of a biological neuron that processes inputs by assigning weights, sums them, applies an activation function, and produces an output.

3.

McCulloch–Pitts Neuron Model:
The McCulloch–Pitts neuron is a simple computational model in which inputs are weighted and summed, and the neuron produces an output of **1** if the sum exceeds a fixed threshold; otherwise, the output is **0**.

It uses a **binary step activation function** and does not support learning.

4.

Thresholding Logic in Neural Networks:
Thresholding logic is a method where a neuron produces an output based on whether its input sum crosses a specific **threshold**:

$$\text{Output} = \begin{cases} 1, & \text{if weighted sum} \geq \text{threshold} \\ 0, & \text{if weighted sum} < \text{threshold} \end{cases}$$

It is used in **binary decision-making neurons**, like in the McCulloch–Pitts model.

5.

Linear Perceptron:
A linear perceptron is a type of artificial neuron that computes a **weighted sum of its inputs** and passes it through a **linear activation function** (or threshold function) to produce an output.

It is mainly used for **linearly separable problems**.

6.

Linear Separability:
A problem is **linearly separable** if its data points can be separated into classes using a **straight line (in 2D), plane (in 3D), or hyperplane (in higher dimensions)**.

✅ Example: AND and OR logic gates are linearly separable, but XOR is **not**.

7.

Perceptron Decision Function:

$$y = \begin{cases} 1, & \text{if } \sum_{i=1}^{n} w_i\, x_i + b \geq 0 \\ 0, & \text{if } \sum_{i=1}^{n} w_i\, x_i + b < 0 \end{cases}$$

Where:

- x_i= input

- w_i= weight of input
- b= bias (threshold)
- y= output

8.

Convergence Theorem of Perceptron Learning Algorithm:

If a set of training data is **linearly separable**, the perceptron learning algorithm is guaranteed to **converge to a set of weights** that correctly classifies all training examples in a **finite number of steps**.

9.

Problem Addressed by Skip Connections in ResNet:

Skip connections in ResNet solve the **vanishing gradient problem** in very deep neural networks.

- In deep networks, gradients can become **very small** during backpropagation, making training slow or ineffective.
- Skip connections **bypass one or more layers** by adding the input directly to the output, allowing **gradients to flow easily** and enabling training of much deeper networks.

10.

Differences between ResNet and DenseNet:

Feature	ResNet	DenseNet
Connection Type	Uses **skip (residual) connections** that add input to output	Uses **dense connections** that **concatenate** all previous layer outputs
Feature Reuse	Less direct reuse of all previous features	Promotes **maximum feature reuse** from all preceding layers

Feature	ResNet	DenseNet
Network Depth Efficiency	Easier to train very deep networks	Requires fewer parameters due to feature reuse but has more memory usage

5 marks

1.

Biological Neuron Model

A biological neuron is a nerve cell that processes and transmits information in the nervous system. It has three main components:

1. Dendrites – Receive signals from other neurons.
2. Cell Body (Soma) – Integrates the received signals.
3. Axon – Transmits the processed signal to other neurons via synapses.

Working:

- Signals received by dendrites are summed in the soma.
- If the total input exceeds a certain threshold, the neuron “fires,” sending an electrical impulse along the axon.

Inspiration for Artificial Neurons

Artificial neurons, or computational neurons, are modeled after biological neurons:

- Inputs (x_i) → resemble dendrites receiving signals
- Weights (w_i) → represent the strength of each input
- Summation and activation function → mimics soma integrating signals and firing if threshold is exceeded
- Output (y) → analogous to the axon transmitting the signal

This model forms the basis of perceptrons and deep neural networks

2.

McCulloch–Pitts Neuron Model (M-P Neuron)

The **McCulloch–Pitts (M-P) neuron** is a **simplest computational model** of a biological neuron, proposed by Warren McCulloch and Walter Pitts in 1943. It is used to model **logic operations** in neural networks.

Components of M-P Neuron:

1. **Inputs ($x_1, x_2, \ldots, x_n$)** – Signals received from other neurons.
2. **Weights ($w_1, w_2, \ldots, w_n$)** – Each input is multiplied by a weight representing its importance.
3. **Summation Function** – Computes the total weighted input:

$$S = \sum_{i=1}^{n} w_i \, x_i$$

4. **Threshold (θ)** – A fixed value that decides if the neuron fires.
5. **Output (y)** – The neuron fires (**y = 1**) if the total input exceeds the threshold; otherwise, it does not (**y = 0**).

Working Rule:

$$y = \begin{cases} 1, & \text{if } \sum w_i x_i \geq \theta \\ 0, & \text{if } \sum w_i x_i < \theta \end{cases}$$

Thresholding Logic

Thresholding logic means that the **output of the neuron is binary (0 or 1)** depending on whether the **weighted sum of inputs reaches the threshold**.

Example 1: AND Gate

- Inputs: $x_1, x_2 \in \{0,1\}$
- Weights: $w_1 = w_2 = 1$
- Threshold: $\theta = 2$

x1 x2 Σwixi Output y

0 0 0 0

x1	x2	Σwixi	Output y
0	1	1	0
1	0	1	0
1	1	2	1

Example 2: OR Gate

- Weights: $w_1 = w_2 = 1$
- Threshold: $\theta = 1$

x1	x2	Σwixi	Output y
0	0	0	0
0	1	1	1
1	0	1	1
1	1	2	1

3.

Linear Perceptron Model

The **Linear Perceptron** is a type of artificial neuron used for **linearly separable classification problems**. It is inspired by the biological neuron and McCulloch–Pitts model.

Components:

1. **Inputs ($x_1, x_2, \ldots, x_n$)** – Features from the dataset.

2. **Weights ($w_1, w_2, \ldots, w_n$)** – Importance assigned to each input.
3. **Bias (b)** – Threshold value to control the decision boundary.
4. **Summation Function:**

$$S = \sum_{i=1}^{n} w_i \, x_i + b$$

5. **Activation Function:** A **step function** (binary output):

$$y = \begin{cases} 1, & S \geq 0 \\ 0, & S < 0 \end{cases}$$

Working:

- The perceptron calculates the weighted sum of inputs.
- If the sum exceeds the threshold (bias), the neuron fires (**y = 1**); otherwise, it outputs 0.
- It is trained using the **Perceptron Learning Algorithm** to adjust weights for correct classification.

Linear Separability

A dataset is **linearly separable** if a **straight line (2D), plane (3D), or hyperplane (n-D)** can separate the data points of different classes.

Examples:

1. **AND Gate (Linearly Separable)**

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Illustration:

y=1 * (1,1)

y=0 * (0,0), (0,1), (1,0)

A straight line can separate the output **1** from **0**. ✅

2. **XOR Gate (Not Linearly Separable)**

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

No straight line can separate outputs 1 and 0. ❌

Illustration:

y=1 * (0,1), (1,0)

y=0 * (0,0), (1,1)

4.

Perceptron Learning Algorithm (PLA)

The **Perceptron Learning Algorithm** is used to train a linear perceptron by adjusting its weights to correctly classify linearly separable data.

Step-by-Step Algorithm

1. **Initialize Weights and Bias:**
   - Set all weights $w_i = 0$(or small random values).
   - Set bias $b = 0$(or a small value).
2. **For each training sample** (x, y)**:**
   - Compute the **output of the perceptron**:

$$\hat{y} = \begin{cases} 1, & \sum w_i x_i + b \geq 0 \\ 0, & \sum w_i x_i + b < 0 \end{cases}$$

3. **Update Weights and Bias if Output is Wrong:**
   - If $\hat{y} \neq y$:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$
$$b \leftarrow b + \eta(y - \hat{y})$$

        - η= learning rate (a small positive constant)

4. **Repeat:**
    - Go through all training samples repeatedly until all are correctly classified (or maximum iterations reached).

Convergence Behavior

- **Convergence Theorem:**
    - If the training data is **linearly separable**, the perceptron learning algorithm is **guaranteed to converge** in a **finite number of steps**.
    - It will find a **set of weights and bias** that correctly classifies all samples.
- **Non-Linearly Separable Data:**
    - If the data is **not linearly separable**, PLA **does not converge**; it will keep updating weights endlessly.
    - Solutions: use **multi-layer perceptrons (MLP)** or **non-linear activation functions**.

5.

Convergence Theorem of Perceptron Learning Algorithm (PLA)

Statement:
If the training dataset is **linearly separable**, the Perceptron Learning Algorithm **will converge** to a set of weights and bias that correctly classify all training samples in a **finite number of iterations**.

Meaning:

- "Converge" means the **weight updates stop** because all samples are classified correctly.
- The theorem guarantees that the PLA **can find a solution** for linearly separable data.

Significance

1. **Guaranteed Solution:** For linearly separable data, PLA is **reliable** and will find a working model.
2. **Foundation for Neural Networks:** Establishes the concept of **iterative learning by weight adjustment**.
3. **Simplicity:** The algorithm is easy to implement and understand for beginners.

Limitations

1. **Only for Linearly Separable Data:**
    - If the data is not linearly separable (e.g., XOR), PLA **does not converge**.

2. **Choice of Learning Rate (η) Matters:**
    - Too large → overshooting; too small → slow convergence.
3. **No Optimal Solution for Non-Separable Data:**
    - PLA keeps updating weights endlessly if classes overlap.
4. **Cannot Handle Complex Patterns:**
    - Limited to **single-layer, linear classification**; cannot model non-linear relationships.

6.

1. ResNet (Residual Network)

Architecture:

- Introduced to train **very deep neural networks**.
- Uses **residual blocks** with **skip connections**:
    - Input to a layer is **added directly** to its output.
    - Helps preserve information across layers.

Block Diagram of Residual Block:

Input x ---> [Conv + BN + ReLU] ---> [Conv + BN] ---> + ----> Output y

|

x (skip connection)

Working Principle:

- Each residual block learns a **residual function**:

$$F(x) = H(x) - x \implies H(x) = F(x) + x$$

where $H(x)$is the desired mapping, $x$is input, $F(x)$is the residual function.

- Skip connections help **gradients flow directly** during backpropagation, solving the **vanishing gradient problem**.

Advantages over traditional networks:

1. Can train **very deep networks** (hundreds of layers).
2. Reduces **vanishing gradient problem**.
3. Improves **accuracy and convergence**.

---

2. DenseNet (Densely Connected Network)

Architecture:

- Each layer receives input from **all preceding layers** via **concatenation**.
- Promotes **feature reuse** and strengthens gradient flow.

Block Diagram of Dense Block:

x0 ---> L1 ---> x1

| \

|--------> Concatenate ---> L2 ---> x2

| \

|------------------> Concatenate ---> ...

Working Principle:

- Each layer has access to **all previous feature maps**, not just the immediate previous layer.
- This reduces **redundancy** and allows **compact networks** with fewer parameters.

Advantages over traditional networks:

1. Promotes **maximum feature reuse**, improving efficiency.
2. Requires **fewer parameters** than similarly deep traditional CNNs.
3. Improves **gradient flow**, making training easier.